

An Industrial Experience on the Application of Distributed Testing in an Agile Software Development Environment

Eliane Collins

Nokia Institute of Technology
Manaus, AM, Brazil
eliane.collins@indt.org.br

Gisele Macedo

Nokia Institute of Technology
Manaus, AM, Brazil
gisele.macedo@indt.org.br

Nayane Maia

Federal University of Amazonas
Manaus, AM, Brazil
nayane.maia@icom.ufam.edu.br

Arilo Dias-Neto

Federal University of Amazonas
Manaus, AM, Brazil
arilo@icom.ufam.edu.br

Abstract—Software engineering is by nature a highly collaborative activity. However, this collaboration is more difficult when the teams are geographically separated, as several factors, such as work-time, cultural differences, communication, technical capability, among others, may impact on its success. Moreover, each activity in the software development process has specific needs in a distributed software development (DSD) environment. In this paper, we report an industrial experience of a testing team separated geographically in the context of a software project that followed an agile method. The agile testing process was adapted to deal with the DSD characteristics. As results, we observed the feasibility and success of applying distributed testing in an agile software project when dealing with aspects such as communication, coordination, collaboration, task distribution, and tests automation. The challenges, results, and lessons learned from this experience are discussed.

Keywords—Computer-supported collaborative work; agile testing; SCRUM.

I. INTRODUCTION

In recent years, Software Engineering has been adapted to the evolutions occurring in the global software development scenario. In this context, two evolutions could be highlighted and they are the focus of this paper: the distributed software development (DSD) and agile methods. These have recently been combined, so that agile methods are used in geographically distributed contexts [1].

Analyzing the DSD scenario, we can observe the decentralization of software development, distributing efforts among teams geographically separated [2].

The benefits achieved through the adoption of the DSD in software projects could be measured in ways, ranging from reduced development cost, leveraging time-zone effectiveness, cross-site modularization of development work, access to large skilled labor pool, innovation and best shared practices, and closer proximity to market and customer [3]. However, DSD introduces new elements that need to be attended to reach the success of a distributed software project, separated, according to [4], in: strategic, cultural, inadequate communication, knowledge management, project management, and technical issues. Thus, the challenge is to balance the benefits and drawbacks while aiming to maximize the productivity in a software project. When we analyze the agile methods, they are based on the four values published in the agile manifesto [5]. The agile methods define software engineering practices to be followed aiming at the development of software systems with higher productivity and quality. In the context of agile method, the testing activities

are called agile testing. The current main agile methods would be *Scrum* [7], *XP (extreme Programming)* [8], and *Crystal* [9].

Both scenarios (DSD and agile methods) describe testing as an essential element to reach the software quality. However, some aspects are dealt with in a different way during testing (e.g.: process, documentation, and tasks allocation). Testing consists in an experimental evaluation with the purpose of revealing failures during the execution of a software system [6].

This paper describes an industrial experience on the application of distributed testing in a software development environment following the agile project management method Scrum. We report the adaptation in the testing process to make feasible the distributed tests in an agile environment, the solutions implemented to reduce the impact of communication issues and the difficulties observed in the allocation of tasks in the software project.

This paper is organized as follows: Section 2 discusses the application of testing in the agile and the DSD environments according to the technical literature, contextualizing the scenario where this experience was conducted. Section 3 presents the application of distributed testing in a software project following an agile method, its adaptations and decisions. Section 4 describes some results and lessons learned from this experience. Finally, section 5 presents the conclusions and further steps.

II. BACKGROUND

A. Agile Testing

As described in [10], agile testing does not just mean testing on agile projects but testing an application with a plan to learn about it, let the customer information guide the testing in line with agile values working software responding to change.

Testing Automation is considered the key to agile development and the core to agile testing. According to [11], different tests have different purposes, hence a diagram of agile testing quadrants describing how each quadrant reflects different reasons to test, is presented in Figure 1.

- (Q1) Technology Facing tests in Support of Programming (Technical requirement testing – Does this method do what the developer intended?)
- (Q2) Business Facing tests in Support of Programming (Business requirements tests – Does the code do what it should?)
- (Q3) Business Facing tests to Criticize the Product

(Business defect testing – Does the code do something it shouldn't? Are there missing requirements?)

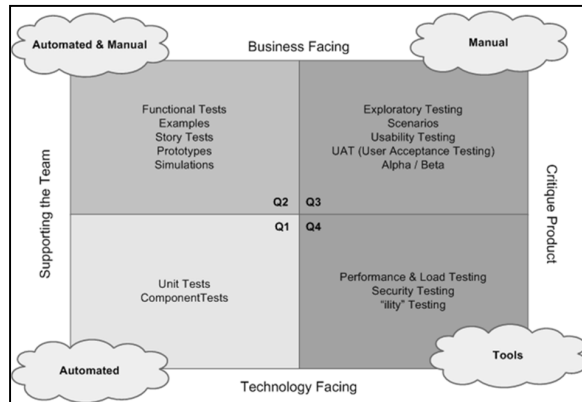


Figure 1. Agile Test Quadrants [11]

- **(Q4)** Technology Facing tests to Criticize the Product (Technical defect testing – Are there leaks? Can it handle a load? Is it fast enough?)

Traditional software testing focuses almost exclusively on the right side (Q3 and Q4), criticizing the product but not playing a productive part in supporting the creation of the product (Q1 and Q2) [11]. Moreover, it is involved late in the development process to detect failures but not to prevent them. In agile testing, the team is not only involved in identifying, but also in preventing failures by regular interaction with developers and customers.

Thus, agile testing is a challenge to testers who are used to working in traditional waterfall projects mainly because they do not have to wait for system delivery at the end of the project to start the test activities but they need to be proactive and start the test tasks from the very beginning of the project together with the developers as one team and being successful in this challenge requires experience, overcoming barriers and mind set. In our experience, we applied some concepts of agile testing with a geographically distributed testing team and we reported the challenges and lessons observed in this project.

B. Testing in DSD

In [12], the authors describe a study revealing that outsourced-offshored software testing is the second largest outsourced activity, after coding. According to the interviewed testing team members, working with manual and automated tests, the testing would be structured in three high-level phases: test preparation, test execution, and client management.

- The test preparation phase begins when the testing team receives the functional requirements and overlaps with the development phase. In general, the main activity in this phase is the assessment of the feasibility of completing the tasks within the planned time, reallocating the tasks, or requesting additional resources.
- After releasing a stable software version, the execution phase is started. Some interviewed testing teams mentioned that as soon as this phase starts, they experience pressure and urgency to finish testing activities as soon as possible.
- When the testers approve the product, based on satisfactory test results, the client-management phase begins.

The results of this study suggested that for the manual test engineers, the preparation phase was considered relatively shorter when compared to the opinion of automated test engineers, but the execution phase would be considerably longer. However, for the automated test engineers, the preparation phase would be longer because it takes considerable effort to create the automated test scripts from the manual tests stated in natural language, but the execution phase would be relatively shorter.

In another study [13], the authors observed the testing activities in an outsourcing software project. In the distributed project, modules were implemented and unit tested at remote development centers, and then integrated periodically at a central location. The challenges reported by authors were: a) how to do effective unit-testing at a remote site where test data cannot be directly accessed is entirely due to data privacy and replication issues, b) how to smoothen the integration of modules developed at separate locations and reduce integration errors, and c) how to test a software system using imprecise interface documents. In a distributed software project, it would be indeed worth the extra effort in making interface requirements and other specification documents convey the precise and full information needed for product integration. This can significantly enhance the quality of unit testing at any site, and also reduce the gap between unit and subsequent integration testing.

In [14], the authors describe a survey that characterized research regarding software development according to five aspects: 1) Agile Software Development; 2) Distributed Software Development; 3) Product Domain; 4) Product Criticality Security; 5) Amount of tests executed by participant. As results, the authors concluded: (1) the participants working with agile software development are not happy with the usual practice of test-first (probably because they were starting the use of this method); (2) the “non-agile” participants do not know the agile testing practices; (3) the absence of infrastructure is considered an important factor to delay the software deployment; (4) modifications in the source-code should increase the delay and the complexity during integration testing; (5) the participants working with distributed software development were disturbed with modification in the source-code while they were performing integration testing.

Besides characterizing the scenario of testing in distributed software development, these works are related to the scenario where testing is performed in an environment where the development team is distributed, and the software produced needs to be tested. However, they do not cover the scenario where the development is performed in a same site and the testing activities are distributed among different teams geographically. In our project, we needed to conduct testing using two teams geographically separated, which introduced new challenges and results, as described in the next section.

III. APPLYING AGILE TESTING IN A DSD ENVIRONMENT

A. Software Project Characteristics

This project had been conducted in the context of a research institute in Manaus/Amazonas/Brazil aiming at developing a web application to manager advertising campaigns, called NAP. This software project was developed following the Scrum agile methodology [7], and it consisted of 30 stories (Product Backlog) divided into 9 Sprints (iterations)

varying from 2 to 3 weeks.

This project had a development team composed of 1 Scrum Master and 3 full-time developers. The Testing Team, the focus of this paper, consisted of 6 professionals, being 2 full-time testers located in the same physical environment of the developers and four part-time testers who worked in another site (geographically separated) as a result of a cooperation between the research institute and the Federal University of Amazonas (UFAM).

The NAP project was developed using a web platform (PHP + MySQL database), and IDE Eclipse. The system was composed of 8 screens (4 forms to register campaigns and users and 4 screens to search and generate reports).

B. Testing Process

The NAP's software testing process followed the Scrum methodology, where the testing team was integrated to the project team, participating in Scrum ceremonies (sprint review, daily, retrospective and planning meetings). Their responsibilities were to plan test cases through the stories described in the Product Backlog, specify the acceptance criteria, and use test automation tools to speed up the execution activities of each sprint.

Due to the geographic distribution of the testing team, the testing process was designed considering the characteristics of a DSD (distributed software development) environment. We needed to provide a testing structure allowing access to the project's information by all team members. Thus, in order to attend this demand, the testing team used a server dedicated to be connected remotely with the following tools:

- **TestLink¹**: used to manage test plans, write test cases, and report tests execution. Besides, the selection of tests to compose a test suite was done manually, *TestLink* acted as an editor and organizer of test cases, storing all information. It facilitated the creation of test plans and reports documentation, and the controlling of the tests execution versions;
- **Mantis Bug Tracker²**: through this tool, which was already used in the institute, the tester registered the defects found, sent them to the developers and controlled the lifecycle of each defect.
- **FireScrum³**: a web tool for Scrum taskboard was used to detail tasks defined for the project sprint. This tool was used for both testing teams (remote and local) to record and update the tasks progress. Thus, the test coordinator used these as data in the project daily meeting.
- **Subversion⁴**: it was used to share and manage information among the test team members.

Moreover, in order to ensure the operation of this infrastructure and facilitate communication among the teams, a Test Leader located in the institute was responsible for defining the testing tasks, planning the sprint activities and reporting the progress of these activities at the daily meetings. Only the Test Leader could communicate with the development team, avoiding a larger communication network

in the testing team. The remote testing team was allocated to the tasks of designing, automated (creating scripts), and executing test cases for the stories developed in each sprint.

The tasks of test process during the sprint were incremental and iterative following the scrum process. They are represented in Figure 2. The dark rectangles represent the Test Leader's activities and the lighter rectangles represent the Remote Testing team's activities.

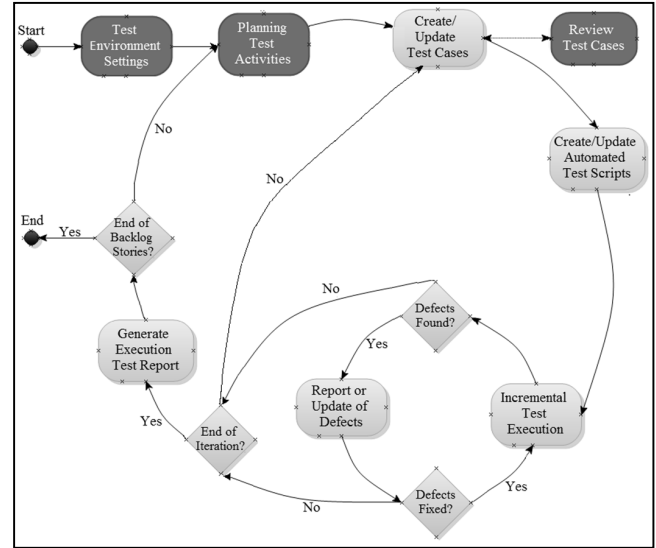


Figure 2. Overview of NAP's Testing Process

The task *Incremental Test Execution* includes execution of exploratory tests and automation of regression tests. For NAP project, the tool chosen for the functional tests automation was the *Selenium RC* and *IDE⁵*. Selenium uses the approach record-play support and tests web applications for the browser Mozilla Firefox. *Selenium IDE* can record the user actions in the browser, creating test scripts in several programming languages and executing them later. The *Selenium Java API* used in the project allows running the test scripts in other browsers such as the versions of Internet Explorer. The automated test suite is updated and executed in every sprint. The tools *TestLink* and *Selenium* were integrated, thus, when test cases are executed in the Selenium tool, their results are recorded automatically in *TestLink*.

C. Communication Process

Contrasting to classic process, the agile software development process is distinguished mainly due to its ability to rapidly accommodate changes in the original requirements and give priority to the development of functionalities through executable code instead of output extensive written documentation, and also because of its quick responses to change and collaboration with the customer rather than following rigid plans and contract negotiations [5]. According to agile practices, communication is an essential factor for the success of the agile project. Project members who have good communication process can cooperate more and mitigate risks of changes during the project. It is an essential factor when some teams of the project are geographically distributed.

¹ TestLink URL: <http://www.teamst.org/>

² Mantis URL: <http://www.mantisbt.org/>

³ FireScrum URL: <http://www.firescrum.org/>

⁴ Subversion URL: <http://subversion.tigris.org/>

⁵ Selenium URL: <http://seleniumhq.org/>

To work with distributed teams, it is important to use tools to facilitate communication. In this project we used several tools like e-mail, a free scrum tool for task board (*FireScrum*), and the chat and video-conference tool (Skype) to make possible online communication among team's members, resolving doubts as soon as possible. Thus, e-mail was used only for offline communication (invitations to meetings and project documents).

Daily, all team members needed to access the online *FireScrum* tool, where the tasks were created and allocated to the testing team members by the Test Leader. The Remote Testing team was responsible for updating the *FireScrum* tasks every day, reporting the progress and impediments.

Figure 3 represents the communication flow among the project teams. According to the flow, communication between the Development Team and the Test Leader was straight (face-to-face) while communication with the Remote Testing team occurred through the server test tools.

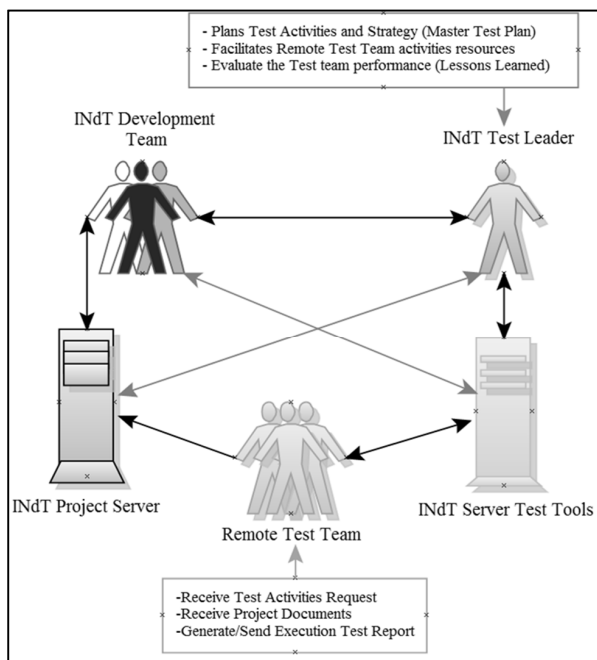


Figure 3. NAP's Communication Network.

D. Tasks Allocation

Using the Scrum methodology, tasks are defined and estimated in *Sprint Planning Meeting*. In this ceremony, each professional involved must participate to specify the tasks necessary to accomplish the stories. All members were invited to this event, including the remote team. On presentation of the definition and clearly understanding the tasks, the Test Leader inserted them into the *FireScrum* tool and allocated them to the testing team members. During the sprint, the Remote Testing team could add and assign new tasks, when appropriate.

With all communication tools available, any possible impediment from the remote testing team could have been quickly detected and easily solved avoiding delays. Each professional team was responsible for his/her external tasks and he/she had to communicate any eventual difficulty as soon as possible. The task status could be defined as: *to do* (to be

made), *in progress* (ongoing task) and *done* (tasks completed). Every day, before the daily meeting of the project in the company, the Test Leader checked the progress of tasks to inform the development team.

Among the main tasks assigned to the Remote Testing team, we can cite: specification of new test cases, update of test cases from previous stories, creation and automation of test scripts, execution of test cases and test scripts of the stories in sprint, defects registration, execution of automated scripts to regression tests and validation of solved defects.

The main Test Leader's tasks can be summarized in: review created test cases, monitor tasks in *FireScrum*, facilitate test tasks, send project information and report test execution.

E. Planning and Restrospective Meetings

Following Scrum, there are two other important meetings to be performed: planning and retrospective. Planning is the meeting where the whole team decides to express their opinion about how to get the stories according to the prioritized backlog. In NAP project, all testing team participated in this meeting (including the remote team), choosing the stories to be developed and estimating the complexity of the selected stories. The retrospective meeting is considered a very important ceremony because it is where is said what went right and what should be improved for the next sprint. All team members, including the remote testing team, participated in this event, sharing experiences and understanding the difficulties that would be improved in the next sprint. With the participation of the remote testing team in both Scrum ceremonies, it's possible to get the unit of the project team and everybody feels as part of the team.

IV. CHALLENGES AND LESSONS LEARNED

In this experience on conducting distributed testing in an agile software project, we could observe it can work very well. However, some issues need to be managed to avoid the risks introduced by the combination of these software engineering practices (DSD + agile practices). Thus, we identified some challenges and key lessons that we learned for minimizing the impact of the geographical distance between the testing teams:

A. Communication and Coordination are essential factors for the success of distributed testing

All main research works in the DSD field indicate communication and coordination as important aspects to carry the success in a distributed software project, and we could also confirm these claims. Some practices were essential to reach this success:

- Allocating one person (test leader) as a link between the local and remote testing team is very important to avoid a large communication network and, consequently noises. Thus, all information and decisions would always pass through this professional, responsible for distributing information, solving impedances and communication problems, and making the testing tasks easier.
- A communication protocol should be formalized, regulating how the teams should keep contact with each other. That includes structure of emails, communication tools, meetings, timetable, and so on. This is essential to avoid losing data, effort, and quality in the software project. For instance, the definition of a standard for test cases specification and bug reporting allows a tester to

complete a task started by another tester or validate a bug reported by another professional.

- An online chat tool should be used and the key persons in the project (Scrum master, development and test leaders) should be always available to clarify doubts.
- Periodic meetings between the development and testing teams should be scheduled. This contributes to understand the complex user's stories. Moreover, the physical presence of the remote testing team in the Scrum meetings (Planning, Retrospective and Review) is very important to share, among all software project members, information regarding problems, suggestion of improvements and planning of new activities.

B. *The project information should be available with details to all members*

As part of the testing team is separated geographically, only short stories and acceptance criteria descriptions are not enough for test cases' specification. Thus, user's scenarios and the application wireframe should be available to cover all features to be developed/tested, bypassing the limitations imposed by the distance between part of the testers and the product owner.

Moreover, changes in the user's scenarios result in a high effort to update test cases and their automation scripts. Therefore, these scenarios should be kept updated, otherwise they can affect the quality of the testing activity.

C. *Automation reduces the needs of physical presence in the testing process*

The tests automation has an essential role in the success of distributed testing, because both teams would be able to run the regression tests suite constantly without depending on time zones between the remote and local testing teams. Moreover, automation speeds up the time required for running the tests even when new releases are published by the development team close to the deployment deadline.

D. *Supporting tools are always important, but testing team organization is more*

Tools are also highlighted in the technical literature as an important element in a distributed software project. In the context of distributed testing in an agile project, two special tools should be cited:

- A Scrum dashboard tool presents all activities performed daily by the remote testing team, showing information regarding the performance of testers and the tasks' progress. Thus, the test leader is provided with information to support the testing process management, mitigating eventual risks. In our software project we used the *FireScrum* tool.
- A test management tool controls all information regarding the testing activities, test cases specification and validation, creation of new releases, test running, number of detected failures, and bug tracking (creation, correction, and validation). In our software project we used the *TestLink* and *Mantis* tools.

Besides their importance, tools can fail, and we need to be prepared for this moment. If the network or a server crashes, all remote teams' activities can be affected because communication between the teams will cease to exist. Thus, we cannot be dependent on tools to perform our activities. In this

case, once more the test leader has a fundamental role to maintain the continuity of the testing process and team organization as it's exposed in Section II-D.

V. CONCLUSIONS

This paper presented our experience in performing distributed testing in an agile software development environment. We could observe that it is feasible to integrate the DSD with the agile practices, maintaining management and organization of a software project, because some of them are considered in the testing activities for both scenarios, as the need of an efficient communication, automation as a resource to reduce cost, and task allocation in small parts. On the other hand, the difference in these scenarios regarding the testing activities, such as continuous integration and daily meetings, could be avoided with some technological solutions already reported in the technical literature.

Some challenges and lessons learned could be extracted from these experience and they could support other software engineers when performing testing in a similar environment. As next steps, we are evaluating the effectiveness of the distributed testing activities in new software projects when compared to the scenario where testing is performed in only one site by software testing metrics like testing effort, number of detected failures pre and post-delivery, and schedule adherence.

ACKNOWLEDGMENT

The authors thank INCT-SEC and INdT for you contribution to this work.

REFERENCES

- [1] Paasivaara, M., Durasiewicz, S. & Lassenius, C.; Using scrum in a globally distributed project: a case study. *Software Process: Improvement and Practice*, 13, 527–544, 2008.
- [2] Carmel, E.; Prikladnicki, R.; Does Time Zone Proximity Matter for Brazil? A Study of the Brazilian I.T. Industry (July 22, 2010). Available at SSRN: <http://dx.doi.org/10.2139/ssrn.1647305>.
- [3] Conchúir, E.; Ågerfalk, P.; Olsson, H.; Fitzgerald, B; Global software development: where are the benefits?. *Commun. ACM* 52, 8 (August 2009), 127-131. DOI=10.1145/1536616.1536648.
- [4] Herbsleb, J.; Moitra, D.; Global Software Development. *IEEE Software* 18(2), 2001.
- [5] Beck, K.; et al. (2001). "Manifesto for Agile Software Development". Agile Alliance. Retrieved February 2012.
- [6] Kaner, C., Exploratory Testing, Florida Institute of Technology, Quality Assurance Institute Worldwide Annual Software Testing Conference Orlando, FL, Novembro. (2006).
- [7] Schwaber K, Beedle M; Agile software development with scrum[M]. New Jersey:Prentice Hall,2001, pp. 29–51.
- [8] Beck K. Extreme Programming Explained: Embrace Change, Addison Wesley, 2000.
- [9] Cockburn, A.; Crystal Clear a Human-Powered Methodology for Small Teams (First ed.), Addison-Wesley Professional, 2004.
- [10] Crispin, L.; Gregory, J.; Agile Testing: A Practical Guide for Testers and Agile Teams, Addison-Wesley, 2009, ISBN 0-321-53446-8.
- [11] Marick, B. ; Everyday Scripting with Ruby: For Teams, Testers, and You; Pragmatic Bookshelf, 2007; ISBN 0-9776166-1-4.
- [12] Shah, H.; Sinha, S.; Harrold, M.J.; Outsourced, Offshored Software-Testing Practice: Vendor-Side Experiences. *ICGSE 2011*: 131-140.
- [13] Sengupta, B.; Chandra, S.; Sinha, V.; A research agenda for distributed software development. In *Proceedings of the 28th international conference on Software engineering (ICSE '06)*. ACM, New York, NY, USA, 731-740, 2006. DOI=10.1145/1134285.1134402.
- [14] Causevic, A., Sundmark, D., & Punnekkat, S. (2010). An Industrial Survey on Contemporary Aspects of Software Testing. 2010 Third International Conference on Software Testing, Verification and Validation, 393-401. IEEE. doi:10.1109/ICST.2010.52.